REVISION CHEAT SHEET: CONCURRENT / IMMUTABLE

Concurrent and Immutable Collections in C# and .NET

Interview-ready notes · examples · C# · diagrams · clickable index

INDEX / TABLE OF CONTENTS

Click a section name to jump to that section.

1. Why Ordinary Collections Are Not Enough
Core Mental Model
2. Thread-Safe Collection Does Not Mean Thread-Safe Application
Example
3. Collection-Level Atomicity vs Business-Level Atomicity
4. Main Concurrent Collection Types
Part 1 — ConcurrentDictionary<TKey,TValue>
5. Why ConcurrentDictionary Exists
6. TryAdd
7. TryGetValue
8. TryRemove
9. TryUpdate
10. GetOrAdd
Critical Consequence
11. Expensive One-Time Initialization
12. AddOrUpdate
Important AddOrUpdate Rule
13. ConcurrentDictionary Enumeration
14. Count Can Immediately Become Stale
Part 2 — ConcurrentQueue<T>
15. FIFO Semantics
Queue Flow
16. TryDequeue
17. TryPeek
18. Queue Enumeration Is a Snapshot
Snapshot Diagram
Part 3 — ConcurrentStack<T>
19. LIFO Semantics
Stack Flow
20. Range Operations
Stack Enumeration
Part 4 — ConcurrentBag<T>
21. What Is a Bag?
Good Mental Model
Do Not Use It as a Queue
Bag Enumeration
22. Choosing Queue, Stack, Bag, or Dictionary
Part 5 — Producer/Consumer Collections
23. Producer/Consumer Problem
24. Why ConcurrentQueue Alone May Not Be Enough
25. BlockingCollection<T>
Example
Bounded Capacity
Completion
Important Modern Limitation
26. Channel<T> — Modern Async Producer/Consumer Alternative
Bounded Channel
BlockingCollection vs Channel
Part 6 — Immutable Collections
27. What Does Immutable Mean?
Immutable Mental Model
28. Main Immutable Types
29. Immutable Does Not Mean Deeply Immutable
30. ImmutableArray<T>
Update
31. ImmutableList<T>
Performance Note
32. ImmutableDictionary<TKey,TValue>
Snapshot Publishing
33. ImmutableHashSet<T>

34. Immutable Queue and Stack
35. Read-Only Is Not the Same as Immutable
Contrast
IReadOnlyList<T> Is Also Not an Immutability Guarantee
36. Builders
Builder Mental Model
37. When to Use a Builder
38. Atomically Publishing Immutable State
39. ImmutableInterlocked
Side-Effect Warning
Immutable Snapshot Update Flow
40. Concurrent Collection vs Immutable Collection
Comparison
41. When ConcurrentDictionary Is Better
42. When ImmutableDictionary Is Better
43. Immutable Snapshot Pattern
44. Concurrent Collections Still Need Correct Value Design
45. Extension Methods and Thread-Safety Caveat
46. Snapshot vs Atomic View
47. Performance Considerations
Read-Only Scenario
48. ConcurrentDictionary Contention
49. Immutable Allocation Costs
50. ImmutableArray for Read-Heavy Snapshots
51. Builders for Bulk Construction
52. Security Considerations
Better Key
Immutable Security Benefit
53. ASP.NET Core Singleton Services
Scoped Service
54. Concurrent Collections Are Process-Local
55. Common Mistake — Replacing Every Collection with ConcurrentCollection
Common Mistake — ContainsKey Followed by Add
Common Mistake — Assuming GetOrAdd Factory Runs Once
Common Mistake — Assuming AddOrUpdate Delegate Runs Once
Common Mistake — ConcurrentDictionary Makes Values Thread-Safe
Common Mistake — Check Count Then Act
Common Mistake — Using ConcurrentBag When Order Matters
Common Mistake — Busy-Waiting on ConcurrentQueue
Common Mistake — BlockingCollection in Async Web Code
Common Mistake — Assuming ReadOnly Means Immutable
Common Mistake — Forgetting to Assign Immutable Result
Common Mistake — Immutable Collection with Mutable Elements
Common Mistake — Mutable Builder Shared Globally
Common Mistake — Concurrent Writers Reassign Immutable Field Normally
Common Mistake — Assuming Enumeration Is Identical Across Concurrent Collections
Best Practices
Practical Exercise 1 — Concurrent Cache
Practical Exercise 2 — Atomic Status Transition
Practical Exercise 3 — Fix GetOrAdd Side Effects
Practical Exercise 4 — FIFO Work Queue
Practical Exercise 5 — ConcurrentBag Pool
Practical Exercise 6 — Bounded Async Channel
Practical Exercise 7 — Immutable Snapshot
Practical Exercise 8 — Immutable Update
Practical Exercise 9 — Builder
Practical Exercise 10 — Atomic Immutable Update
Practical Exercise 11 — ReadOnly vs Immutable
Practical Exercise 12 — Choose the Right Collection
Exactly 25 Interview Questions
1. What are concurrent collections in .NET?
2. Why can't I simply use List<T> or Dictionary<TKey,TValue> from many threads?
3. Does a thread-safe collection make the objects stored inside it thread-safe?
4. What is ConcurrentDictionary<TKey,TValue> used for?
5. Why is TryAdd better than ContainsKey followed by an add?
6. What is GetOrAdd , and can its value factory execute more than once?
7. Why should GetOrAdd and AddOrUpdate delegates avoid side effects?
8. What does TryUpdate do?
9. What is ConcurrentQueue<T> ?
10. What is ConcurrentStack<T> ?
11. What is ConcurrentBag<T> ?
12. How does enumeration behave for concurrent collections?

13. What is BlockingCollection<T> ?
14. What is the difference between BlockingCollection<T> and Channel<T> ?
15. What is an immutable collection?
16. What are the main immutable collection types?
17. Is an immutable collection deeply immutable?
18. What is the difference between read-only and immutable?
19. Why do immutable collection methods return a new value?
20. What are immutable collection builders?
21. Why can concurrent reassignment of an immutable field still lose updates?
22. What is ImmutableInterlocked ?
23. When should you choose a concurrent collection instead of an immutable collection?
24. Are concurrent collections distributed or process-wide?
25. How would you use concurrent and immutable collections in the Fundraising Platform?
Quick Revision Table
Complete Fundraising Example
A. Concurrent Processing State
B. Async Receipt Queue with Channel
C. Immutable Fund Rules Snapshot
End-to-End Architecture
Collection Decision Flow
Immutable Decision Flow
Producer/Consumer Decision Flow
Final End-to-End Mental Model

CONCURRENT / IMMUTABLE

Topic 29 of 40

When several threads or Tasks share a collection, a normal collection such as:

```C#
List<T>
Dictionary<TKey, TValue>
Queue<T>
Stack<T>
```

is not automatically safe for simultaneous mutation.

For example:

```C#
Dictionary<long, Gift> gifts = [];
```

can work perfectly when one execution path owns it. But if many threads concurrently add, remove, and update values, ordinary `Dictionary<TKey,TValue>` doesn't provide the synchronization required to make those writes safe.

.NET therefore provides two major approaches:

EXAMPLE / FLOW

```
Concurrent collections
    →
Allow controlled concurrent mutation
of one shared collection

Immutable collections
    →
Avoid modifying an existing collection;
changes produce a different collection
```

The `System.Collections.Concurrent` namespace provides specialized thread-safe collections such as `ConcurrentDictionary`, `ConcurrentQueue`, `ConcurrentStack`, `ConcurrentBag`, and `BlockingCollection`. Microsoft recommends these types when multiple threads need to add or remove elements concurrently. Immutable collections provide another strategy: once created, a particular collection value never changes, making it naturally safe to share for reading.

1. Why Ordinary Collections Are Not Enough

CONCURRENT / IMMUTABLE

Suppose several workers process Gifts:

```C#
private readonly Dictionary<long, Gift>
    _cache = [];
```

Thread A:

```C#
_cache.Add(
    1001,
    giftA);
```

Thread B simultaneously:

```C#
_cache.Add(
    1002,
    giftB);
```

The fact that the two keys are different doesn't mean concurrent mutation of a normal `Dictionary<TKey,TValue>` is safe.

Similarly:

```C#
List<Gift> gifts = [];
```

with several threads calling:

```C#
gifts.Add(gift);
```

is not a correct general concurrent design.

The generic collection types in `System.Collections.Generic` don't provide built-in synchronization for concurrent writes; user code must synchronize them or use a collection specifically designed for concurrent access.

Core Mental Model

EXAMPLE / FLOW

```
One owner / one thread mutating
        ↓
List<T>, Dictionary<TKey,TValue>, etc.
can be appropriate
```

EXAMPLE / FLOW

```
Many threads mutating one collection
        ↓
Concurrent collection
or external synchronization
```

EXAMPLE / FLOW

```
Many threads reading shared state
and mutation can be avoided
        ↓
Immutable collection
```

CONCURRENT / IMMUTABLE

2. Thread-Safe Collection Does Not Mean Thread-Safe Application

This is one of the most important rules in this topic.

Suppose:

C#

```
ConcurrentDictionary<long, Gift>
    gifts = new();
```

The dictionary safely coordinates operations on its internal mapping.

That doesn't automatically make this safe:

C#

```
Gift gift = gifts[giftId];

gift.TotalAmount += 100m;
```

if multiple threads are mutating the same `Gift`.

The collection protects:

EXAMPLE / FLOW

```
Key → value mapping
```

It does not automatically protect:

EXAMPLE / FLOW

```
Mutable state inside TValue
```

Example

C#

```
ConcurrentDictionary<long, List<Gift>>
    giftsByCampaign = new();
```

This dictionary can safely store and retrieve the `List<Gift>` references.

But this:

C#

```
List<Gift> campaignGifts = giftsByCampaign[campaignId];

campaignGifts.Add(
    gift);
```

still mutates an ordinary `List<Gift>`.

So:

A concurrent outer collection doesn't recursively make its co

3. Collection-Level Atomicity vs Business-Level Atomicity

Consider:

```
C#
ConcurrentDictionary<long, decimal>
    balances = new();
```

Suppose you need:

```
EXAMPLE / FLOW
Read Fund A balance
Read Fund B balance
Move $100 from A to B
```

Even if every individual dictionary operation is thread-safe, the entire transfer:

```
EXAMPLE / FLOW
Read A

Read B

Update A

Update B
```

is not automatically one atomic transaction.

Another thread can interleave between those operations.

Concurrent collections provide atomicity for **specific collection operations**, not arbitrary multi-step business workflows.

For compound invariants you may still need:

```
EXAMPLE / FLOW
A lock

A transaction

A redesign

Immutable snapshot replacement

Database concurrency controls
```

depending on where the state lives.

4. Main Concurrent Collection Types

The most important types are:

Type	Semantics	Main use
`ConcurrentDictionary<TKey,TValue>`	Key/value	Concurrent lookup/update
`ConcurrentQueue<T>`	FIFO	Producer/consumer queue

Type	Semantics	Main use
`ConcurrentStack<T>`	LIFO	Concurrent stack
`ConcurrentBag<T>`	Unordered	Concurrent pool/bag
`BlockingCollection<T>`	Blocking/bounded producer-consumer wrapper	Synchronous producer-consumer
`IProducerConsumerCollection<T>`	Producer/consumer abstraction	Underlying collection contract

`ConcurrentQueue<T>` and `ConcurrentStack<T>` use lock-free techniques based on `Interlocked`, while other concurrent collections may use combinations of fine-grained synchronization mechanisms.

Part 1 — ConcurrentDictionary<TKey,TValue>

5. Why ConcurrentDictionary Exists

A normal dictionary:

```
C#
Dictionary<long, Gift>
```

is excellent when one execution path owns mutation.

For concurrent access:

```
C#
ConcurrentDictionary<long, Gift>
```

provides thread-safe dictionary operations.

Example:

```
C#
ConcurrentDictionary<long, Gift>
    cache = new();
```

Microsoft documents its public and protected members as safe for concurrent use from multiple threads. Access through certain implemented interfaces or extension methods isn't automatically covered by the same guarantee.

6. TryAdd

```
C#
bool added =
    cache.TryAdd(
        gift.GiftId,
        gift);
```

Behavior:

```
EXAMPLE / FLOW
Key absent
    →
Add item
    →
true
```

EXAMPLE / FLOW

```
Key already exists
    →
Do not replace
    →
false
```

This is much safer than:

C#

```
if (
    !cache.ContainsKey(
        gift.GiftId))
{
    cache[gift.GiftId] = gift;
}
```

Why?

Because the second version performs:

EXAMPLE / FLOW

```
Check
    ↓
time gap
    ↓
Add
```

Another thread can modify the dictionary between the check and update.

`TryAdd` expresses the compound intent as one concurrent collection operation.

7. TryGetValue

C#

```
if (
    cache.TryGetValue(
        giftId,
        out Gift? gift))
{
    Process(
        gift);
}
```

Prefer this when you need:

EXAMPLE / FLOW

```
Check whether key exists
+
retrieve value
```

rather than:

C#

```
if (
    cache.ContainsKey(
        giftId))
{
    Gift gift = cache[giftId];
}
```

Between:

EXAMPLE / FLOW

```
ContainsKey
```

and:

EXAMPLE / FLOW

```
indexer access
```

another thread could remove the entry.

8. TryRemove

C#

```
if (
    cache.TryRemove(
        giftId,
        out Gift? removedGift))
{
    Console.WriteLine(
        $"Removed {removedGift.GiftId}");
}
```

This performs:

EXAMPLE / FLOW

```
Try remove
+
return removed value
```

as one dictionary operation.

9. TryUpdate

Suppose:

C#

```
ConcurrentDictionary<long, GiftStatus>
    statuses = new();
```

You want:

EXAMPLE / FLOW

```
Only change Pending → Completed
if value is still Pending
```

Use:

C#

```
bool updated =
    statuses.TryUpdate(
        giftId,
        GiftStatus.Completed,
        GiftStatus.Pending);
```

Conceptually:

EXAMPLE / FLOW

```
Current value == Pending?
        ↓
Yes
    →
replace with Completed
```

If another thread already changed the status:

EXAMPLE / FLOW

```
TryUpdate
    →
false
```

This is similar to a compare-and-swap operation at the dictionary-entry level.

10. GetOrAdd

A common pattern is:

> ⓘ Get the existing value, or create/add one if missing.

C#

```
GiftSummary summary =
    summaries.GetOrAdd(
        giftId,
        id =>
            BuildSummary(
                id));
```

This looks simple, but there is an important concurrency rule.

The factory:

C#

```
id =>
    BuildSummary(id)
```

may execute more than once when multiple threads race for the same missing key.

`ConcurrentDictionary` invokes `GetOrAdd` factories outside its internal locks, so another thread may insert a value while your factory is running. Only one value becomes the stored entry, but multiple factory invocations may occur.

Critical Consequence

Avoid:

C#

```
var item =
    cache.GetOrAdd(
        giftId,
        id =>
        {
            ChargeCreditCard();

            SendExternalMessage();
```

```
            return CreateValue(
                id);
        });
```

CONCURRENT / IMMUTABLE

because those side effects could happen multiple times.

Prefer factories that are:

EXAMPLE / FLOW

```
Pure

Idempotent

Cheap enough to repeat

or explicitly protected by another design
```

11. Expensive One-Time Initialization

Suppose creating a cached value is very expensive and must only execute once.

One common pattern is:

C#

```
ConcurrentDictionary<long, Lazy<GiftSummary>>
    cache = new();

Lazy<GiftSummary> lazy =
    cache.GetOrAdd(
        giftId,
        id =>
            new Lazy<GiftSummary>(
                () =>
                    BuildSummary(
                        id)));

GiftSummary summary = lazy.Value;
```

Multiple `Lazy<GiftSummary>` wrappers may theoretically be constructed during the race, but callers use the winning value stored in the dictionary.

The expensive work represented by the winning `Lazy<T>` can then use its normal thread-safe initialization behavior.

Use this pattern only when one-time creation actually matters.

12. AddOrUpdate

Example:

C#

```
ConcurrentDictionary<long, int>
    giftCountsByCampaign = new();

int newCount =
    giftCountsByCampaign.AddOrUpdate(
        campaignId,
        addValue:
            1,
        updateValueFactory:
            (
                id,
```

```
                currentCount) =>
                currentCount + 1);
```

CONCURRENT / IMMUTABLE

Conceptually:

EXAMPLE / FLOW

```
Campaign absent?
    →
add 1

Campaign exists?
    →
replace current value
with current + 1
```

Important AddOrUpdate Rule

Like `GetOrAdd`, the user-provided delegates used by `AddOrUpdate` execute outside the dictionary's internal locks and may execute more than once under contention. Therefore they should not contain side effects that must occur exactly once.

Good:

C#

```
(
    key,
    current) =>
    current + 1
```

Dangerous:

C#

```
(
    key,
    current) =>
{
    SendEmail();

    return current + 1;
}
```

13. ConcurrentDictionary Enumeration

You can enumerate:

C#

```
foreach (
    KeyValuePair<long, Gift> pair
    in cache)
{
}
```

while other threads are reading or writing.

But its enumerator does **not** represent a guaranteed moment-in-time snapshot. It may expose modifications made after enumeration began.

So don't write logic such as:

EXAMPLE / FLOW

```
Enumerate dictionary
    ↓
Assume these entries represent
one perfectly consistent instant
```

CONCURRENT / IMMUTABLE

unless your application separately provides that consistency.

14. Count Can Immediately Become Stale

This is valid:

C#

```
int count = cache.Count;
```

But after the value is returned:

EXAMPLE / FLOW

```
another thread may add/remove immediately
```

Therefore:

C#

```
if (
    cache.Count == 0)
{
    // ...
}
```

doesn't establish that the dictionary will still be empty on the next line.

This isn't a bug in the concurrent collection.

It is simply the reality of concurrent state.

Part 2 — ConcurrentQueue<T>

15. FIFO Semantics

`ConcurrentQueue<T>` is a thread-safe:

EXAMPLE / FLOW

```
First In
First Out
```

collection.

Example:

C#

```
ConcurrentQueue<GiftWorkItem>
    queue = new();
```

Producer:

C#

```
queue.Enqueue(
    item);
```

Consumer:

C#
```
if (
    queue.TryDequeue(
        out GiftWorkItem? item))
{
    Process(
        item);
}
```

All public and protected members of `ConcurrentQueue<T>` are documented as thread-safe.

Queue Flow

EXAMPLE / FLOW
```
Enqueue A
Enqueue B
Enqueue C

Queue:
A → B → C
```

Then:

EXAMPLE / FLOW
```
TryDequeue
    →
A
```

Next:

EXAMPLE / FLOW
```
TryDequeue
    →
B
```

16. TryDequeue

Why:

C#
```
TryDequeue(
    out item)
```

instead of something like:

EXAMPLE / FLOW
```
Check Count
then Remove
```

Because in concurrent code:

EXAMPLE / FLOW
```
Check
    ↓
Another thread changes queue
```

```
    ↓
Act
```

is inherently racy.

The `Try...` pattern combines:

EXAMPLE / FLOW

```
Attempt operation
+
tell caller success/failure
```

17. TryPeek

C#

```
if (
    queue.TryPeek(
        out GiftWorkItem? next))
{
}
```

returns the next value without removing it.

But remember:

EXAMPLE / FLOW

```
Peeked value
    ↓
another consumer may dequeue it
```

before your next operation.

So `TryPeek` should not be treated as reserving the item.

18. Queue Enumeration Is a Snapshot

`ConcurrentQueue<T>` enumeration is different from `ConcurrentDictionary`.

The queue's enumerator represents a moment-in-time snapshot and doesn't reflect later updates. It is safe to enumerate while other threads modify the queue, and enumeration preserves FIFO order for that snapshot.

Snapshot Diagram

At enumeration start:

EXAMPLE / FLOW

```
Queue
A B C
```

Snapshot:

EXAMPLE / FLOW

```
A B C
```

Then another thread enqueues:

EXAMPLE / FLOW

```
D
```

CONCURRENT / IMMUTABLE

Current queue:

EXAMPLE / FLOW

```
A B C D
```

Existing enumeration still reflects:

EXAMPLE / FLOW

```
A B C
```

Part 3 — ConcurrentStack<T>

19. LIFO Semantics

`ConcurrentStack<T>` is thread-safe and follows:

EXAMPLE / FLOW

```
Last In
First Out
```

Example:

C#

```
ConcurrentStack<GiftWorkItem>
    stack = new();
```

Push:

C#

```
stack.Push(
    item);
```

Pop:

C#

```
if (
    stack.TryPop(
        out GiftWorkItem? item))
{
}
```

All public and protected members can be used concurrently.

Stack Flow

EXAMPLE / FLOW

```
Push A
Push B
Push C
```

Stack:

EXAMPLE / FLOW

```
C
B
A
```

CONCURRENT / IMMUTABLE

Then:

EXAMPLE / FLOW

```
TryPop
    →
C
```

20. Range Operations

`ConcurrentStack<T>` also provides useful operations such as:

C#

```
PushRange(...)
TryPopRange(...)
```

when many items need to move together.

These can reduce per-item overhead compared with repeatedly calling:

C#

```
Push
```

or:

C#

```
TryPop
```

for suitable workloads.

Stack Enumeration

`ConcurrentStack<T>` enumeration returns a moment-in-time snapshot and exposes items in LIFO order for that snapshot.

Part 4 — ConcurrentBag<T>

21. What Is a Bag?

A bag is an unordered collection.

It allows:

EXAMPLE / FLOW

```
Duplicates
```

and doesn't promise:

EXAMPLE / FLOW

```
FIFO

LIFO
```

```
Sorted order
```

Example:

```
C#
ConcurrentBag<ReusableBuffer>
    buffers = new();
```

Add:

```
C#
buffers.Add(
    buffer);
```

Take:

```
C#
if (
    buffers.TryTake(
        out ReusableBuffer? buffer))
{
}
```

`ConcurrentBag<T>` is optimized particularly for scenarios where the same thread often both adds and removes items.

Good Mental Model

```
EXAMPLE / FLOW
"I need an available item.
I don't care which one."
```

That makes `ConcurrentBag<T>` useful for scenarios such as:

```
EXAMPLE / FLOW
Object pools

Reusable buffers

Unordered worker-owned items
```

Do Not Use It as a Queue

If business requirements say:

```
EXAMPLE / FLOW
First request submitted
must be first request processed
```

do not use:

```
C#
ConcurrentBag<T>
```

Use a FIFO mechanism instead.

Bag Enumeration

CONCURRENT / IMMUTABLE

`ConcurrentBag<T>` enumeration represents a moment-in-time snapshot and can safely occur while the bag is being modified.

22. Choosing Queue, Stack, Bag, or Dictionary

Ask what relationship the collection must express.

EXAMPLE / FLOW

```
Need key-based lookup?
    →
ConcurrentDictionary
```

EXAMPLE / FLOW

```
Need oldest-added item first?
    →
ConcurrentQueue
```

EXAMPLE / FLOW

```
Need newest-added item first?
    →
ConcurrentStack
```

EXAMPLE / FLOW

```
Don't care about order;
need arbitrary available items?
    →
ConcurrentBag
```

Part 5 — Producer/Consumer Collections

23. Producer/Consumer Problem

A producer creates work:

EXAMPLE / FLOW

```
Producer
    ↓
Work item
    ↓
Buffer
```

A consumer processes work:

EXAMPLE / FLOW

```
Buffer
    ↓
Consumer
```

Example:

EXAMPLE / FLOW

```
API receives Gift export request
        ↓
Producer creates export work item
```

```
        ↓
Queue
        ↓
Background worker
        ↓
Generate export
```

CONCURRENT / IMMUTABLE

24. Why ConcurrentQueue Alone May Not Be Enough

`ConcurrentQueue<T>` lets producers and consumers safely modify a queue.

But suppose the queue is empty.

Consumer:

C#

```
if (
    queue.TryDequeue(
        out GiftWorkItem? item))
{
}
```

returns:

EXAMPLE / FLOW

```
false
```

Now what?

You don't want:

C#

```
while (true)
{
    if (
        !queue.TryDequeue(
            out item))
    {
        continue;
    }
}
```

because that can become a busy loop consuming CPU.

You need some form of:

EXAMPLE / FLOW

```
Waiting / signaling
```

This is where higher-level producer/consumer abstractions become useful.

25. BlockingCollection<T>

`BlockingCollection<T>` wraps an `IProducerConsumerCollection<T>` and adds:

EXAMPLE / FLOW

```
Blocking

Bounding

Completion signaling
```

```
Cancellation-aware operations
```

Its default underlying collection is a `ConcurrentQueue<T>`, though another producer/consumer collection can be supplied.

Example

```C#
using BlockingCollection<GiftWorkItem>
    workItems =
        new(
            boundedCapacity:
                100);
```

Producer:

```C#
workItems.Add(
    item,
    cancellationToken);
```

Consumer:

```C#
GiftWorkItem item =
    workItems.Take(
        cancellationToken);
```

Bounded Capacity

Suppose capacity is:

```EXAMPLE / FLOW
100
```

Producer creates items faster than consumer processes them.

Without a bound:

```EXAMPLE / FLOW
100
1,000
100,000
1,000,000
```

items may accumulate.

With a bound:

```EXAMPLE / FLOW
Producer
    ↓
Collection full
    ↓
Producer blocks
until space exists
```

This is:

or at least a synchronous form of flow control.

Completion

Producer:

```
C#
workItems.CompleteAdding();
```

Consumers can use completion-aware consumption patterns after all producers have finished.

Important Modern Limitation

`BlockingCollection<T>` was designed around blocking producer/consumer operations, not asynchronous `await`-based coordination. Microsoft's current documentation explicitly recommends considering `Channel<T>` for asynchronous producer/consumer scenarios.

26. `Channel<T>` — Modern Async Producer/Consumer Alternative

`Channel<T>` belongs to:

```
C#
System.Threading.Channels
```

rather than:

```
C#
System.Collections.Concurrent
```

but it is highly relevant when designing modern async pipelines.

Producer:

```
C#
await channel.Writer.WriteAsync(
    item,
    cancellationToken);
```

Consumer:

```
C#
await foreach (
    GiftWorkItem item
    in channel.Reader.ReadAllAsync(
        cancellationToken))
{
    await ProcessAsync(
        item,
        cancellationToken);
}
```

Channels are designed specifically for asynchronous producer/consumer communication and support bounded or unbounded capacity. Bounded channels can asynchronously ap
producers outrun consumers.

Bounded Channel

```C#
Channel<GiftWorkItem> channel =
    Channel.CreateBounded<GiftWorkItem>(
        100);
```

Default full behavior for a bounded channel waits asynchronously for space.

This fits an ASP.NET Core/background-service pipeline much better than blocking request threads with a `BlockingCollection`.

BlockingCollection vs Channel

Requirement	Better starting point
Synchronous producer/consumer threads	`BlockingCollection<T>`
`await`-based producer/consumer	`Channel<T>`
Bounded async backpressure	`Channel<T>`
Legacy blocking pipeline	`BlockingCollection<T>`
BackgroundService async worker	Usually `Channel<T>`

Part 6 — Immutable Collections

27. What Does Immutable Mean?

An immutable collection instance cannot be changed after it is created.

Example:

```C#
ImmutableList<long> giftIds =
    ImmutableList.Create(
        1001L,
        1002L);
```

Call:

```C#
ImmutableList<long> updated =
    giftIds.Add(
        1003L);
```

The original:

```C#
giftIds
```

still contains:

EXAMPLE / FLOW

```
1001
1002
```

The new value:

C#

```
updated
```

contains:

EXAMPLE / FLOW

```
1001
1002
1003
```

Immutable collections allow consumers to safely share a collection knowing that its contents won't later be structurally modified. Microsoft describes them as providing implicit thread safety for shared access.

Immutable Mental Model

Mutable:

EXAMPLE / FLOW

```
Collection A
    ↓ Add
Same Collection A changes
```

Immutable:

EXAMPLE / FLOW

```
Collection A
    ↓ Add
Collection B
```

Collection A remains unchanged.

28. Main Immutable Types

Important types include:

EXAMPLE / FLOW

```
ImmutableArray<T>

ImmutableList<T>

ImmutableDictionary<TKey,TValue>

ImmutableHashSet<T>

ImmutableQueue<T>

ImmutableStack<T>

ImmutableSortedDictionary<TKey,TValue>

ImmutableSortedSet<T>
```

The `System.Collections.Immutable` namespace also provides builders and `ImmutableInterlocked` for specialized update scenarios.

29. Immutable Does Not Mean Deeply Immutable

Suppose:

```
C#
ImmutableList<Gift>
    gifts =
        ImmutableList.Create(
            gift);
```

You cannot structurally modify the list itself:

```
EXAMPLE / FLOW
Cannot add/remove
without getting another list
```

But if `Gift` is mutable:

```
C#
gift.Status = GiftStatus.Completed;
```

the object referenced inside the immutable list changed.

Therefore:

> **Immutable collection means the collection structure is immutable. It doesn't automatically make every contained object immutable.**

For strong immutability, prefer immutable element types too, such as immutable records or immutable value objects.

30. ImmutableArray<T>

`ImmutableArray<T>` is particularly useful when you want:

```
EXAMPLE / FLOW
Fixed snapshot

Fast indexed access

Safe sharing

Array-like semantics
```

Example:

```
C#
ImmutableArray<long> fundIds =
[
    10,
    20,
    30
];
```

or explicitly:

```C#
ImmutableArray<long> fundIds =
    ImmutableArray.Create(
        10L,
        20L,
        30L);
```

`ImmutableArray<T>` is a thread-safe immutable array abstraction.

Update

```C#
ImmutableArray<long> updated =
    fundIds.Add(
        40L);
```

Remember:

```C#
fundIds.Add(
    40L);
```

alone does not mutate `fundIds`.

If you ignore the returned value, the change is lost.

31. ImmutableList<T>

```C#
ImmutableList<GiftSummary> gifts =
    ImmutableList<GiftSummary>
        .Empty;
```

Add:

```C#
gifts =
    gifts.Add(
        summary);
```

Remove:

```C#
gifts =
    gifts.Remove(
        summary);
```

The list type itself is thread-safe to share because existing instances are never modified.

Performance Note

`ImmutableList<T>` isn't simply a regular `List<T>` with mutation disabled.

Its data structure has different performance characteristics. For example, Microsoft documents `ImmutableList<T>` indexed access and additions as logarithmic operations, whereas `List<T>` indexed access is constant time and append is normally amortized constant time. Choose immutability for semantics/correctness first, then measure if performance is important.

32. ImmutableDictionary<TKey,TValue>

CONCURRENT / IMMUTABLE

Example:

```C#
ImmutableDictionary<long, GiftSummary>
    summaries =
        ImmutableDictionary<long, GiftSummary>
            .Empty;
```

Add:

```C#
summaries =
    summaries.Add(
        giftId,
        summary);
```

Set:

```C#
summaries =
    summaries.SetItem(
        giftId,
        updatedSummary);
```

Remove:

```C#
summaries =
    summaries.Remove(
        giftId);
```

Each operation returns a new immutable dictionary.

The original remains valid and safe for readers holding the old reference.
`ImmutableDictionary<TKey,TValue>` itself is thread-safe.

Snapshot Publishing

This gives an elegant pattern:

EXAMPLE / FLOW

```
Reader A
    ↓
Snapshot V1

Writer creates
Snapshot V2

Reader B
    ↓
Snapshot V2

Reader A can safely finish
using V1
```

No reader sees:

EXAMPLE / FLOW

```
half-updated dictionary
```

CONCURRENT / IMMUTABLE

because each published instance has a fixed structure.

33. ImmutableHashSet<T>

Use when you need:

EXAMPLE / FLOW

```
Unique elements
+
immutable set semantics
```

Example:

C#

```
ImmutableHashSet<long> authorizedFundIds =
    ImmutableHashSet.Create(
        10L,
        20L,
        30L);
```

Update:

C#

```
authorizedFundIds =
    authorizedFundIds.Add(
        40L);
```

The original set remains unchanged.

This is particularly useful for configuration-style state that:

EXAMPLE / FLOW

```
changes occasionally

is read frequently

must be safe to share
```

34. Immutable Queue and Stack

Immutable versions also exist:

C#

```
ImmutableQueue<T>
ImmutableStack<T>
```

Instead of mutating an existing queue or stack, operations return updated versions.

Conceptually:

EXAMPLE / FLOW

```
Old Queue
    ↓ Enqueue
New Queue
```

The previous queue still represents the old state.

35. Read-Only Is Not the Same as Immutable

CONCURRENT / IMMUTABLE

This distinction is extremely important.

Suppose:

```
C#
List<long> source =
[
    1,
    2
];

ReadOnlyCollection<long> readOnly = source.AsReadOnly();
```

Consumer cannot directly call:

```
C#
readOnly.Add(...)
```

But someone who still has:

```
C#
source
```

can do:

```
C#
source.Add(
    3);
```

and the read-only wrapper reflects that change.

Microsoft explicitly describes `ReadOnlyCollection<T>` as a wrapper: mutations to the underlying list remain visible through the wrapper.

Contrast

Read-only:

EXAMPLE / FLOW

```
"You cannot modify it
through this reference."
```

Immutable:

EXAMPLE / FLOW

```
"This collection instance
cannot change."
```

These are different guarantees.

IReadOnlyList<T> Is Also Not an Immutability Guarantee

An API returning:

```
C#
IReadOnlyList<Gift>
```

says:

EXAMPLE / FLOW

```
Caller cannot mutate through
this interface
```

It does not guarantee that the underlying collection—or the elements inside it—cannot change elsewhere. Microsoft specifically notes that read-only list element content isn't guaranteed to be read-only.

36. Builders

Repeated immutable operations can produce more overhead than constructing state mutably and publishing one final immutable value.

Builders help.

Example:

C#

```
ImmutableArray<GiftSummary>.Builder
    builder =
        ImmutableArray
            .CreateBuilder<GiftSummary>();
```

Build:

C#

```
foreach (
    Gift gift
    in gifts)
{
    builder.Add(
        new GiftSummary(
            gift.GiftId,
            gift.TotalAmount));
}
```

Publish:

C#

```
ImmutableArray<GiftSummary>
    result = builder.ToImmutable();
```

Builder Mental Model

EXAMPLE / FLOW

```
Mutable construction phase
        ↓
Builder
        ↓
Finish
        ↓
Immutable snapshot
        ↓
Share safely
```

Builders are construction tools, not the immutable value itself. For example, Microsoft's `ImmutableArray<T>.ToBuilder()` documentation explicitly states that the returned mutable builder isn't thread-safe.

37. When to Use a Builder

Prefer a builder when:

EXAMPLE / FLOW

```
Many changes are made together

Only final state needs to be published

Repeated immutable replacements would be unnecessarily expensive
```

Example:

C#

```
ImmutableDictionary<long, GiftSummary>.Builder
    builder =
        ImmutableDictionary
            .CreateBuilder<long, GiftSummary>();

foreach (
    Gift gift
    in gifts)
{
    builder[gift.GiftId] =
        Map(
            gift);
}

ImmutableDictionary<long, GiftSummary>
    snapshot = builder.ToImmutable();
```

38. Atomically Publishing Immutable State

Immutability makes individual collection values safe to share.

But suppose many writers update this field:

C#

```
private ImmutableList<GiftSummary>
    _summaries = ImmutableList<GiftSummary>.Empty;
```

This isn't safe:

C#

```
_summaries =
    _summaries.Add(
        summary);
```

under concurrent writers.

Why?

Two threads may both read version 1:

EXAMPLE / FLOW

```
Thread A reads V1
Thread B reads V1
```

A creates:

B creates:

Then:

A's update may be lost.

39. ImmutableInterlocked

.NET provides:

```C#
ImmutableInterlocked
```

for atomic update patterns over immutable values.

Example:

```C#
private ImmutableList<GiftSummary>
    _summaries = ImmutableList<GiftSummary>.Empty;

public void AddSummary(
    GiftSummary summary)
{
    ImmutableInterlocked.Update(
        ref _summaries,
        current =>
            current.Add(
                summary));
}
```

`ImmutableInterlocked.Update` uses optimistic retry semantics: if another thread wins the update race, the transformation may run again against newer state until the update succeeds.

Side-Effect Warning

Because the transformation may be retried, don't write:

```C#
ImmutableInterlocked.Update(
    ref _summaries,
    current =>
    {
        SendEmail();

        return current.Add(
            summary);
    });
```

The side effect might run more than once.

Keep the transformation:

EXAMPLE / FLOW

```
Deterministic

Side-effect free

Based only on current immutable state
```

Immutable Snapshot Update Flow

EXAMPLE / FLOW

```
Current V1
    ↓
Read
    ↓
Create V2
    ↓
Compare-and-swap
    ├─ success → publish V2
    └─ lost race
        ↓
      reread latest
        ↓
      create new version
        ↓
      retry
```

40. Concurrent Collection vs Immutable Collection

Suppose many threads need to update a cache continuously.

A:

C#

```
ConcurrentDictionary<TKey,TValue>
```

may be ideal.

Suppose state changes rarely but is read thousands of times:

C#

```
ImmutableDictionary<TKey,TValue>
```

with atomic snapshot replacement may be easier to reason about.

Comparison

Concern	Concurrent Collection	Immutable Collection
Existing instance changes	Yes	No
Concurrent mutation	Supported by API	Not applicable to instance
Read sharing	Safe	Safe
Update style	Mutate shared collection	Create new value

Concern	Concurrent Collection	Immutable Collection
Reasoning model	Shared synchronized state	Snapshots
Good for frequent writes	Often	Depends
Good for read-heavy snapshots	Good	Excellent
Multi-step invariants	May still need synchronization	May atomically replace whole snapshot
Element mutability	Still matters	Still matters

41. When ConcurrentDictionary Is Better

Use it when:

EXAMPLE / FLOW

```
Many threads repeatedly add/update/remove
independent keys
```

Example:

EXAMPLE / FLOW

```
Gift processing status cache
```

C#

```
ConcurrentDictionary<long, GiftProcessingState>
    states = new();
```

Workers can independently update keys without rebuilding the whole map.

42. When ImmutableDictionary Is Better

Use it when:

EXAMPLE / FLOW

```
Configuration changes rarely

Readers are extremely frequent

Readers need stable snapshots

Whole-state replacement is natural
```

Example:

EXAMPLE / FLOW

```
Fund routing rules
```

A service can hold:

C#

```
ImmutableDictionary<long, FundRule>
```

and periodically replace the entire snapshot when configuration changes.

Readers never need to lock to inspect the published snapshot.

43. Immutable Snapshot Pattern

CONCURRENT / IMMUTABLE

```C#
private ImmutableDictionary<long, FundRule>
    _rules =
        ImmutableDictionary<long, FundRule>
            .Empty;
```

Reader:

```C#
ImmutableDictionary<long, FundRule>
    snapshot = _rules;

if (
    snapshot.TryGetValue(
        fundId,
        out FundRule? rule))
{
}
```

Writer:

```C#
ImmutableDictionary<long, FundRule>
    updated = BuildRules();

Interlocked.Exchange(
    ref _rules,
    updated);
```

Now each reader gets either:

EXAMPLE / FLOW

```
Old complete snapshot

or

New complete snapshot
```

not an intermediate mutation.

44. Concurrent Collections Still Need Correct Value Design

Bad:

```C#
ConcurrentDictionary<long, Gift>
    gifts = new();
```

Then multiple threads:

```C#
gifts[giftId]
    .AddAllocation(
        allocation);
```

If `Gift` isn't thread-safe:

EXAMPLE / FLOW

```
Dictionary safety
    ≠
Gift safety
```

CONCURRENT / IMMUTABLE

Better designs might use:

EXAMPLE / FLOW

```
Immutable Gift snapshots

One owner per Gift

A lock around Gift invariant

Database transaction/concurrency

Message queue serialization
```

depending on the requirement.

45. Extension Methods and Thread-Safety Caveat

A subtle framework rule: even when a concurrent collection's own public/protected members are thread-safe, operations invoked through implemented interfaces or extension methods aren't universally guaranteed to preserve the same thread-safety guarantees.

For example:

C#

```
concurrentDictionary
    .Where(...)
    .ToList();
```

is a LINQ operation composed over enumeration.

That may be perfectly acceptable, but you should understand its enumeration semantics rather than assuming:

EXAMPLE / FLOW

```
"The whole LINQ pipeline is one
atomic concurrent operation."
```

It isn't.

46. Snapshot vs Atomic View

Suppose:

C#

```
Gift[] snapshot = queue.ToArray();
```

You now have a copied snapshot.

Other threads may continue changing the queue, but your array doesn't change.

Compare that with:

```C#
foreach (
    KeyValuePair<long, Gift> item
    in dictionary)
```

whose enumerator is safe but not guaranteed to represent one frozen instant.

Always know what kind of consistency your operation requires.

47. Performance Considerations

Concurrent collections include synchronization machinery.

Therefore:

```EXAMPLE / FLOW
Concurrent collection
```

is not automatically faster than:

```EXAMPLE / FLOW
normal collection
```

When there is:

```EXAMPLE / FLOW
one thread
```

or:

```EXAMPLE / FLOW
only concurrent readers
```

a normal generic collection may be faster because it doesn't require concurrent-write coordination.

Microsoft specifically notes that synchronization introduces overhead and recommends choosing concurrent collections based on actual multi-threaded requirements rather than automatically replacing generic collections.

Read-Only Scenario

Suppose this dictionary is built once during startup:

```C#
Dictionary<long, FundRule>
```

and never mutated afterward.

Multiple threads can safely read it when no thread is modifying it.

You don't necessarily need:

```C#
ConcurrentDictionary
```

just because the application has multiple threads.

Another good option may be:

C#

```
ImmutableDictionary
```

CONCURRENT / IMMUTABLE

if the immutable guarantee itself is valuable.

48. ConcurrentDictionary Contention

`ConcurrentDictionary` is designed to scale better than a single global external lock for many concurrent update scenarios.

But heavy contention for:

EXAMPLE / FLOW

```
the same key
```

can still serialize logically competing operations.

Example:

C#

```
counts.AddOrUpdate(
    sameCampaignId,
    1,
    (_, current) =>
        current + 1);
```

executed by thousands of workers still concentrates activity on one logical entry.

Thread-safe does not mean contention-free.

49. Immutable Allocation Costs

Immutable operations conceptually return new collection values.

Implementations are designed to reuse existing internal structure where possible rather than naively copying every element on every operation, but immutable operations still have different allocation/performance characteristics than mutable `List<T>` or `Dictionary<TKey,TValue>`.

Microsoft's collection guidance documents, for example, that `ImmutableList<T>` and `ImmutableDictionary<TKey,TValue>` have different asymptotic characteristics than their mutable counterparts.

Use:

EXAMPLE / FLOW

```
Immutable semantics
```

because they improve correctness/design.

Then measure hot paths.

50. ImmutableArray for Read-Heavy Snapshots

For:

EXAMPLE / FLOW

```
Build once

Read many times
```

```
Index frequently
```


`ImmutableArray<T>` can be an excellent choice.

Example:

```C#
ImmutableArray<GiftRule>
    rules =
        BuildRules()
            .ToImmutableArray();
```

Many threads can safely iterate the same snapshot without locks.

51. Builders for Bulk Construction

Bad pattern for a large build:

```C#
ImmutableList<GiftSummary> result = ImmutableList<GiftSummary>.Empty;

foreach (
    Gift gift
    in gifts)
{
    result =
        result.Add(
            Map(
                gift));
}
```

This may be completely fine for small data.

For a large construction pipeline, a builder can be clearer and potentially more efficient:

```C#
ImmutableList<GiftSummary>.Builder
    builder =
        ImmutableList
            .CreateBuilder<GiftSummary>();

foreach (
    Gift gift
    in gifts)
{
    builder.Add(
        Map(
            gift));
}

ImmutableList<GiftSummary> result = builder.ToImmutable();
```

Builders are designed for efficient batches of changes before producing an immutable result.

52. Security Considerations

Concurrent state can accidentally become cross-request or cross-tenant state.

Example:

C#

```
private static readonly
    ConcurrentDictionary<long, Gift>
    _gifts = new();
```

CONCURRENT / IMMUTABLE

If keys are only:

EXAMPLE / FLOW

```
GiftId
```

but IDs are tenant-relative rather than globally unique, one tenant could overwrite or read another tenant's cache entry.

The collection is thread-safe.

The design is still insecure.

Better Key

Potentially:

C#

```
public readonly record struct GiftCacheKey(
    long TenantId,
    long GiftId);
```

Then:

C#

```
ConcurrentDictionary<GiftCacheKey, GiftSummary>
    cache = new();
```

Thread safety and tenant isolation are different concerns.

Immutable Security Benefit

Immutable snapshots can reduce certain accidental mutation bugs:

EXAMPLE / FLOW

```
Authorization rules snapshot
is published
    ↓
Readers can't silently mutate it
```

But immutability doesn't verify:

EXAMPLE / FLOW

```
Who is allowed to read

Which Tenant owns the data

Whether data should be cached
```

Security rules remain explicit application responsibilities.

53. ASP.NET Core Singleton Services

A singleton service can be called concurrently by many requests.

Therefore:

CONCURRENT / IMMUTABLE

C#

```
public sealed class GiftCache
{
    private readonly Dictionary<long, Gift>
        _cache = [];
}
```

is dangerous if request handlers mutate `_cache` concurrently.

Possible solutions include:

EXAMPLE / FLOW

```
ConcurrentDictionary

Immutable snapshot

Explicit lock

External distributed cache

Redesign so singleton has no mutable state
```

The best choice depends on the behavior.

Scoped Service

A typical request-scoped service often has one logical request owner.

That can reduce the need for concurrent collections.

But don't assume:

EXAMPLE / FLOW

```
Scoped
=
cannot experience concurrency
```

because a request can itself start concurrent work.

The actual access pattern still matters.

54. Concurrent Collections Are Process-Local

A:

C#

```
ConcurrentDictionary
```

coordinates threads inside one application process.

Suppose the application is scaled:

EXAMPLE / FLOW

```
Server A
    →
ConcurrentDictionary A

Server B
```

```
    →
ConcurrentDictionary B
```

These are separate dictionaries.

Updates aren't automatically shared.

So use distributed state infrastructure when the business requires cross-instance consistency:

EXAMPLE / FLOW

```
Database

Distributed cache

Message broker

Distributed coordination
```

A concurrent collection isn't a distributed cache.

55. Common Mistake — Replacing Every Collection with ConcurrentCollection

Bad thinking:

EXAMPLE / FLOW

```
"Application has multiple threads,
therefore every List must become concurrent."
```

Thread-safe collections add synchronization costs and specialized semantics.

Use them only for collections that are genuinely shared and concurrently mutated.

Common Mistake — ContainsKey Followed by Add

Unsafe design:

C#

```
if (
    !cache.ContainsKey(
        key))
{
    cache[key] = value;
}
```

Prefer:

C#

```
cache.TryAdd(
    key,
    value);
```

or:

C#

```
cache.GetOrAdd(
    key,
    factory);
```

depending on intended semantics.

Common Mistake — Assuming GetOrAdd Factory Runs Once

It may run multiple times under contention.

Therefore:

```
C#
cache.GetOrAdd(
    id,
    _ =>
    {
        SendPayment();
        return result;
    });
```

is dangerous.

Common Mistake — Assuming AddOrUpdate Delegate Runs Once

Same problem.

The add/update delegates may execute repeatedly while concurrent threads race.

Keep them side-effect free.

Common Mistake — ConcurrentDictionary Makes Values Thread-Safe

It doesn't.

```
C#
ConcurrentDictionary<long, List<Gift>>
```

does not make:

```
C#
List<Gift>
```

safe for concurrent mutation.

Common Mistake — Check Count Then Act

```
C#
if (
    queue.Count > 0)
{
    queue.TryDequeue(
        out _);
}
```

The `Count` check adds little value because another consumer may change the queue immediately.

Simply:

```
C#
if (
    queue.TryDequeue(
        out GiftWorkItem? item))
```

```
{
}
```


Common Mistake — Using ConcurrentBag When Order Matters

`ConcurrentBag<T>` is unordered.

Use a queue when FIFO matters.

Common Mistake — Busy-Waiting on ConcurrentQueue

Bad:

```C#
while (
    true)
{
    if (
        queue.TryDequeue(
            out GiftWorkItem? item))
    {
        Process(
            item);
    }
}
```

This can burn CPU while no work exists.

Use a signaling or producer/consumer abstraction.

For modern async pipelines, `Channel<T>` is usually the better fit.

Common Mistake — BlockingCollection in Async Web Code

`BlockingCollection<T>` deliberately blocks producers/consumers.

Don't use it merely because you need a thread-safe queue inside an async ASP.NET Core pipeline.

For asynchronous producer/consumer scenarios, Microsoft explicitly points toward Channels.

Common Mistake — Assuming ReadOnly Means Immutable

```C#
ReadOnlyCollection<T>
```

can reflect mutations made through the underlying collection reference.

Read-only access and immutable state are different concepts.

Common Mistake — Forgetting to Assign Immutable Result

Wrong:

```C#
rules.Add(
    newRule);
```

if:

```C#
rules
```

is immutable.

Correct:

```C#
rules =
    rules.Add(
        newRule);
```

The mutation-like method returns another collection.

Common Mistake — Immutable Collection with Mutable Elements

```C#
ImmutableList<Gift>
```

doesn't make each `Gift` immutable.

Prefer immutable element types when a full snapshot guarantee matters.

Common Mistake — Mutable Builder Shared Globally

A builder exists to make a mutable construction phase efficient.

Don't publish it as the shared state you're trying to make immutable.

Build:

```EXAMPLE / FLOW
Mutable Builder
```

then publish:

```EXAMPLE / FLOW
Immutable Collection
```

Common Mistake — Concurrent Writers Reassign Immutable Field Normally

This can lose updates:

```C#
_snapshots =
    _snapshots.Add(
        snapshot);
```

when several threads do it concurrently.

Use:

```EXAMPLE / FLOW
ImmutableInterlocked

lock
```

```
single-writer ownership
```

depending on the design.

Common Mistake — Assuming Enumeration Is Identical Across Concurrent Collections

It isn't.

For example:

EXAMPLE / FLOW

```
ConcurrentQueue / ConcurrentStack /
ConcurrentBag
    →
snapshot-style enumeration
```

while:

EXAMPLE / FLOW

```
ConcurrentDictionary
    →
safe concurrent enumeration,
but not a moment-in-time snapshot
```

Best Practices

Choose the collection based on both:

EXAMPLE / FLOW

```
Data semantics
+
Concurrency semantics
```

Use `ConcurrentDictionary` for genuinely concurrent key/value access.

Prefer `TryGetValue`, `TryAdd`, `TryUpdate`, and `TryRemove` over multi-step:

EXAMPLE / FLOW

```
check then act
```

logic.

Keep `GetOrAdd` and `AddOrUpdate` factories free of non-repeatable side effects.

Remember that collection thread safety doesn't make mutable values thread-safe.

Use `ConcurrentQueue` for FIFO work.

Use `ConcurrentStack` for LIFO work.

Use `ConcurrentBag` only when ordering doesn't matter.

Use `BlockingCollection` only when blocking/bounded synchronous producer-consumer behavior fits the requirement.

For new async producer/consumer pipelines, strongly consider `Channel<T>`.

Prefer immutable collections when readers benefit from stable snapshots and updates can create/publish a new version.

Use builders when constructing many immutable elements in one phase.

Use `ImmutableInterlocked` or another synchronization strategy when multiple writers replace immutable state concurrently.

CONCURRENT / IMMUTABLE

Distinguish:

EXAMPLE / FLOW

```
Read-only

Immutable

Thread-safe

Atomic

Distributed
```

because they mean different things.

Don't introduce a concurrent collection where a normal local collection is simpler.

Practical Exercise 1 — Concurrent Cache

Create:

C#

```
ConcurrentDictionary<long, GiftSummary>
    cache = new();
```

Add only if absent:

C#

```
bool added =
    cache.TryAdd(
        summary.GiftId,
        summary);
```

Retrieve:

C#

```
if (
    cache.TryGetValue(
        giftId,
        out GiftSummary? summary))
{
}
```

Remove:

C#

```
cache.TryRemove(
    giftId,
    out _);
```

Practical Exercise 2 — Atomic Status Transition

C#

```
ConcurrentDictionary<long, GiftStatus>
    statuses = new();

bool changed =
```

```
    statuses.TryUpdate(
        giftId,
        GiftStatus.Completed,
        GiftStatus.Pending);
```

CONCURRENT / IMMUTABLE

Question:

What happens if another worker changed it to `Cancelled` first?

Answer:

EXAMPLE / FLOW

```
TryUpdate returns false.
```

Practical Exercise 3 — Fix GetOrAdd Side Effects

Bad:

C#

```
GiftSummary summary =
    cache.GetOrAdd(
        giftId,
        _ =>
        {
            SendReceipt();

            return BuildSummary();
        });
```

Better:

C#

```
GiftSummary summary =
    cache.GetOrAdd(
        giftId,
        _ =>
            BuildSummary());

await SendReceiptAsync(
    summary,
    cancellationToken);
```

Now the external side effect isn't hidden inside a factory that may execute more than once.

Practical Exercise 4 — FIFO Work Queue

C#

```
ConcurrentQueue<GiftWorkItem>
    queue = new();

queue.Enqueue(
    first);

queue.Enqueue(
    second);

if (
    queue.TryDequeue(
        out GiftWorkItem? item))
{
    Process(
```

```
        item);
}
```

First dequeued:

EXAMPLE / FLOW

```
first
```

Practical Exercise 5 — ConcurrentBag Pool

C#

```
ConcurrentBag<GiftFormatter>
    formatters = new();

GiftFormatter Rent()
{
    if (
        formatters.TryTake(
            out GiftFormatter? formatter))
    {
        return formatter;
    }

    return new GiftFormatter();
}

void Return(
    GiftFormatter formatter)
{
    formatters.Add(
        formatter);
}
```

Use this only if pooling is truly justified and object reuse is safe.

Practical Exercise 6 — Bounded Async Channel

C#

```
Channel<GiftWorkItem> channel =
    Channel.CreateBounded<GiftWorkItem>(
        100);
```

Producer:

C#

```
await channel.Writer.WriteAsync(
    workItem,
    cancellationToken);
```

Consumer:

C#

```
await foreach (
    GiftWorkItem item
    in channel.Reader.ReadAllAsync(
        cancellationToken))
{
    await ProcessAsync(
        item,
```

```
        cancellationToken);
}
```


This gives async producer/consumer coordination with bounded capacity.

Practical Exercise 7 — Immutable Snapshot

```C#
ImmutableArray<GiftSummary> summaries =
    gifts
        .Select(
            gift =>
                new GiftSummary(
                    gift.GiftId,
                    gift.TotalAmount))
        .ToImmutableArray();
```

Share:

```C#
summaries
```

with many readers.

No reader can structurally modify that collection instance.

Practical Exercise 8 — Immutable Update

```C#
ImmutableDictionary<long, GiftSummary>
    summaries =
        ImmutableDictionary<long, GiftSummary>
            .Empty;

summaries =
    summaries.SetItem(
        summary.GiftId,
        summary);
```

The previous dictionary remains unchanged.

Practical Exercise 9 — Builder

```C#
ImmutableArray<GiftSummary>.Builder
    builder =
        ImmutableArray
            .CreateBuilder<GiftSummary>();

foreach (
    Gift gift
    in gifts)
{
    builder.Add(
        new GiftSummary(
            gift.GiftId,
            gift.TotalAmount));
}
```

```
ImmutableArray<GiftSummary>
    snapshot = builder.ToImmutable();
```

Practical Exercise 10 — Atomic Immutable Update

```
C#
private ImmutableList<GiftSummary>
    _summaries =
        ImmutableList<GiftSummary>
            .Empty;

public void Add(
    GiftSummary summary)
{
    ImmutableInterlocked.Update(
        ref _summaries,
        current =>
            current.Add(
                summary));
}
```

The update function can be retried, so keep it side-effect free.

Practical Exercise 11 — ReadOnly vs Immutable

```
C#
List<long> original =
[
    1,
    2
];

ReadOnlyCollection<long> view = original.AsReadOnly();

original.Add(
    3);
```

Question:

Does `view` now contain 3?

Answer:

```
EXAMPLE / FLOW
Yes.
```

It is a read-only wrapper over mutable underlying state.

Practical Exercise 12 — Choose the Right Collection

Requirement:

```
EXAMPLE / FLOW
Concurrent key/value cache
```

Choose:

```
EXAMPLE / FLOW
ConcurrentDictionary
```

Requirement:

EXAMPLE / FLOW
FIFO work items

Choose:

EXAMPLE / FLOW
ConcurrentQueue
or Channel for async pipeline

Requirement:

EXAMPLE / FLOW
Unordered object pool

Choose:

EXAMPLE / FLOW
ConcurrentBag

Requirement:

EXAMPLE / FLOW
Stable read-only configuration snapshot

Choose:

EXAMPLE / FLOW
ImmutableDictionary / ImmutableArray

Requirement:

EXAMPLE / FLOW
Multiple writers updating one immutable field

Choose:

EXAMPLE / FLOW
ImmutableInterlocked
or another explicit synchronization strategy

Exactly 25 Interview Questions

1. What are concurrent collections in .NET?

Concurrent collections are collection types designed to support safe concurrent access and mutation by multiple threads.

The main types are in:

```
C#
System.Collections.Concurrent
```

and include `ConcurrentDictionary`, `ConcurrentQueue`, `ConcurrentStack`, `ConcurrentBag`, and `BlockingCollection`.

2. Why can't I simply use `List<T>` or `Dictionary<TKey,TValue>` from many threads?

CONCURRENT / IMMUTABLE

The generic mutable collection types don't provide built-in synchronization for concurrent writes.

If multiple threads mutate them simultaneously, application code must provide synchronization or use a collection designed for concurrent operations.

3. Does a thread-safe collection make the objects stored inside it thread-safe?

No.

For example:

```
C#
ConcurrentDictionary<long, List<Gift>>
```

protects dictionary operations, but the contained `List<Gift>` is still an ordinary mutable list.

Thread safety doesn't automatically propagate recursively into stored objects.

4. What is `ConcurrentDictionary<TKey,TValue>` used for?

It provides a thread-safe key/value collection for scenarios where multiple threads need to read, add, remove, or update entries concurrently.

Its public/protected collection members are designed for concurrent use.

5. Why is `TryAdd` better than `ContainsKey` followed by an add?

This:

```
C#
if (
    !dictionary.ContainsKey(
        key))
{
    dictionary[key] = value;
}
```

contains a race between:

EXAMPLE / FLOW

```
check
and
update
```

`TryAdd` represents the conditional add as one concurrent collection operation.

6. What is `GetOrAdd`, and can its value factory execute more than once?

`GetOrAdd` returns the existing value for a key or attempts to add a new value.

Its factory may execute more than once when threads race for the same key because the factory runs outside the dictionary's internal locks. Only one value becomes the stored entry.

7. Why should `GetOrAdd` and `AddOrUpdate` delegates avoid side effects?

Those delegates can execute multiple times under contention.

A delegate that:

EXAMPLE / FLOW

```
Charges money

Sends email

Publishes a message
```

could therefore perform the side effect multiple times.

Keep such delegates deterministic and preferably side-effect free.

8. What does TryUpdate do?

`TryUpdate` replaces a value only when its current value equals an expected comparison value.

Conceptually:

EXAMPLE / FLOW

```
if current == expected
    replace
else
    fail
```

This is useful for compare-and-update scenarios such as state transitions.

9. What is ConcurrentQueue<T>?

It is a thread-safe FIFO collection.

Typical operations are:

C#

```
Enqueue

TryDequeue

TryPeek
```

Its public and protected members can be used concurrently from multiple threads.

10. What is ConcurrentStack<T>?

It is a thread-safe LIFO collection.

Typical operations include:

C#

```
Push

TryPop

TryPeek
```

It also supports range operations such as `PushRange` and `TryPopRange`.

11. What is ConcurrentBag<T>?

`ConcurrentBag<T>` is an unordered thread-safe collection that supports duplicate values.

It is optimized particularly for scenarios where the same thread frequently adds and removes values, making it useful for some object-pooling scenarios.

12. How does enumeration behave for concurrent collections?

The behavior depends on the collection.

`ConcurrentQueue`, `ConcurrentStack`, and `ConcurrentBag` provide moment-in-time snapshot enumeration.

`ConcurrentDictionary` can be enumerated safely during concurrent modification, but its enumeration isn't a guaranteed moment-in-time snapshot and may reflect modifications made after enumeration began.

13. What is BlockingCollection<T>?

`BlockingCollection<T>` is a thread-safe producer/consumer wrapper that adds:

EXAMPLE / FLOW

```
Blocking

Bounded capacity

Completion signaling

Cancellation support
```

over an `IProducerConsumerCollection<T>`. Its default underlying collection is `ConcurrentQueue<T>`.

14. What is the difference between BlockingCollection<T> and Channel<T>?

`BlockingCollection<T>` is built around synchronous blocking producer/consumer operations.

`Channel<T>` is designed for asynchronous producer/consumer communication using APIs such as:

C#

```
WriteAsync

ReadAsync

ReadAllAsync
```

Microsoft recommends considering Channels for async producer/consumer scenarios.

15. What is an immutable collection?

An immutable collection is a collection whose existing instance cannot be structurally changed.

Operations such as:

C#

```
Add

Remove

SetItem
```

return another collection value while leaving the original unchanged.

Immutable collections are naturally safe to share for concurrent reads.

16. What are the main immutable collection types?

Common types include:

```
ImmutableArray<T>

ImmutableList<T>

ImmutableDictionary<TKey,TValue>

ImmutableHashSet<T>

ImmutableQueue<T>

ImmutableStack<T>

ImmutableSortedDictionary<TKey,TValue>

ImmutableSortedSet<T>
```

17. Is an immutable collection deeply immutable?

No.

The collection structure cannot change, but objects stored inside the collection may themselves be mutable.

For example:

```
C#
ImmutableList<Gift>
```

doesn't prevent mutation of a mutable `Gift` instance stored in that list.

18. What is the difference between read-only and immutable?

Read-only usually means the current reference/API does not permit mutation.

It does not necessarily prevent another reference from changing the underlying collection.

`ReadOnlyCollection<T>`, for example, reflects changes made to the wrapped collection. An immutable collection instance itself cannot be structurally changed.

19. Why do immutable collection methods return a new value?

Because the existing collection cannot change.

Therefore:

```
C#
items.Add(
    value);
```

returns the updated collection rather than modifying `items`.

You normally write:

```
C#
items =
    items.Add(
        value);
```

when the variable should point to the new version.

20. What are immutable collection builders?

Builders are mutable construction helpers designed for efficiently applying many changes before producing an immutable result.

Typical pattern:

EXAMPLE / FLOW

```
Create builder

Perform mutations

Call ToImmutable

Publish immutable result
```

The mutable builder itself should not be treated as the shared immutable state.

21. Why can concurrent reassignment of an immutable field still lose updates?

Consider:

C#

```
_items =
    _items.Add(
        item);
```

Two threads may both read the same old immutable collection, independently create two updated versions, and then overwrite each other's field assignment.

Immutability makes each collection value safe; it doesn't automatically make the variable holding the current value atomically update itself.

22. What is `ImmutableInterlocked`?

`ImmutableInterlocked` provides atomic update operations for immutable collection/state scenarios.

For example:

C#

```
ImmutableInterlocked.Update(
    ref _items,
    current =>
        current.Add(
            item));
```

uses optimistic retry semantics so concurrent writers don't silently overwrite one another. The transformation can be retried and should therefore avoid non-repeatable side effects.

23. When should you choose a concurrent collection instead of an immutable collection?

Choose a concurrent collection when many threads must frequently mutate a shared collection directly.

Choose an immutable collection when stable snapshots, safe sharing, and easier reasoning are more important and updates can naturally create/publish new versions.

24. Are concurrent collections distributed or process-wide?

They are in-process objects.

A `ConcurrentDictionary` running on Server A doesn't automatically synchronize with a `ConcurrentDictionary` on Server B.

Cross-instance consistency requires distributed infrastructure such as a database, distributed cache, or messaging/coordination system.

25. How would you use concurrent and immutable collections in the Fundraising Platform?

A practical mapping is:

EXAMPLE / FLOW

```
Frequently updated in-memory Gift cache
    →
ConcurrentDictionary

FIFO background work
    →
Channel<T> for modern async pipelines

Reusable unordered objects
    →
ConcurrentBag

Stable Fund/Campaign rules snapshot
    →
ImmutableDictionary / ImmutableArray

Multiple writers publishing immutable state
    →
ImmutableInterlocked

SQL/database invariants
    →
Database transactions/concurrency controls,
not in-process collections
```

The most important question is not simply:

EXAMPLE / FLOW

```
"Which collection is thread-safe?"
```

but:

> **What consistency, ordering, ownership, mutation, and lifetime semantics does this data actually require?**

Quick Revision Table

Type / Concept	Main Meaning
`ConcurrentDictionary<TKey,TValue>`	Concurrent key/value access
`TryAdd`	Add only if key absent
`TryGetValue`	Retrieve without check-then-act race
`TryUpdate`	Conditional compare/update
`TryRemove`	Remove and return value

Type / Concept	Main Meaning
`GetOrAdd`	Existing or create/add
`AddOrUpdate`	Add or compute replacement
`ConcurrentQueue<T>`	Concurrent FIFO
`ConcurrentStack<T>`	Concurrent LIFO
`ConcurrentBag<T>`	Concurrent unordered bag
`BlockingCollection<T>`	Blocking producer/consumer
`Channel<T>`	Async producer/consumer
`ImmutableArray<T>`	Immutable array snapshot
`ImmutableList<T>`	Immutable list
`ImmutableDictionary<TKey,TValue>`	Immutable key/value map
`ImmutableHashSet<T>`	Immutable unique set
Builder	Efficient mutable construction phase
`ImmutableInterlocked`	Atomic immutable-state updates
Read-only	Cannot mutate through current API
Immutable	Existing collection instance cannot change
Snapshot	Stable view/version
Atomic operation	One indivisible logical update
Thread-safe	Correct concurrent use under documented operations

Complete Fundraising Example

Suppose the platform needs an in-memory service with three responsibilities:

EXAMPLE / FLOW

```
1. Track processing state for Gifts

2. Queue receipt work

3. Publish a stable snapshot of Fund routing rules
```

Each problem needs a different collection.

A. Concurrent Processing State

C#

```
public enum GiftProcessingState
{
    Pending,
    Processing,
    Completed,
    Failed
}

public sealed class GiftProcessingTracker
{
```

```
    private readonly
        ConcurrentDictionary<long, GiftProcessingState>
        _states = new();

    public bool TryStart(
        long giftId)
    {
        return _states.TryUpdate(
            giftId,
            GiftProcessingState.Processing,
            GiftProcessingState.Pending);
    }

    public void Register(
        long giftId)
    {
        _states.TryAdd(
            giftId,
            GiftProcessingState.Pending);
    }

    public void Complete(
        long giftId)
    {
        _states.AddOrUpdate(
            giftId,
            GiftProcessingState.Completed,
            (
                _,
                _) =>
                GiftProcessingState.Completed);
    }

    public bool TryGetState(
        long giftId,
        out GiftProcessingState state)
    {
        return _states.TryGetValue(
            giftId,
            out state);
    }
}
```

CONCURRENT / IMMUTABLE

Why `ConcurrentDictionary`?

EXAMPLE / FLOW

```
Many requests/workers
    ↓
Independent Gift IDs
    ↓
Frequent key-based updates
```

B. Async Receipt Queue with Channel

Work item:

C#

```
public sealed record ReceiptWorkItem(
    long TenantId,
    long GiftId);
```

Queue:

```C#
public sealed class ReceiptQueue
{
    private readonly
        Channel<ReceiptWorkItem>
        _channel =
            Channel.CreateBounded<ReceiptWorkItem>(
                500);

    public ValueTask QueueAsync(
        ReceiptWorkItem item,
        CancellationToken cancellationToken)
    {
        return _channel.Writer.WriteAsync(
            item,
            cancellationToken);
    }

    public IAsyncEnumerable<ReceiptWorkItem>
        ReadAllAsync(
            CancellationToken cancellationToken)
    {
        return _channel.Reader.ReadAllAsync(
            cancellationToken);
    }
}
```

Producer:

```C#
await receiptQueue.QueueAsync(
    new ReceiptWorkItem(
        tenantId,
        giftId),
    cancellationToken);
```

Background consumer:

```C#
await foreach (
    ReceiptWorkItem item
    in receiptQueue.ReadAllAsync(
        stoppingToken))
{
    await receiptSender.SendAsync(
        item.TenantId,
        item.GiftId,
        stoppingToken);
}
```

This provides an async producer/consumer pipeline and bounded capacity.

C. Immutable Fund Rules Snapshot

```C#
public sealed record FundRule(
    long FundId,
    decimal MinimumAmount,
    bool Active);
```

Service:

```C#
public sealed class FundRuleStore
{
    private ImmutableDictionary<long, FundRule>
        _rules =
            ImmutableDictionary<long, FundRule>
                .Empty;

    public bool TryGet(
        long fundId,
        out FundRule? rule)
    {
        ImmutableDictionary<long, FundRule>
            snapshot = _rules;

        return snapshot.TryGetValue(
            fundId,
            out rule);
    }

    public void Replace(
        IEnumerable<FundRule> rules)
    {
        ImmutableDictionary<long, FundRule>
            snapshot =
                rules.ToImmutableDictionary(
                    rule =>
                        rule.FundId);

        Interlocked.Exchange(
            ref _rules,
            snapshot);
    }
}
```

Reader flow:

```
EXAMPLE / FLOW
Request A
    ↓
reads Snapshot V1
    ↓
uses V1 safely
```

Meanwhile:

```
EXAMPLE / FLOW
Configuration refresh
    ↓
build Snapshot V2
    ↓
atomically publish V2
```

Then:

```
EXAMPLE / FLOW
Request B
    ↓
reads Snapshot V2
```

Neither request observes a partially mutated dictionary.

End-to-End Architecture

EXAMPLE / FLOW

```
HTTP Requests
    ↓
GiftProcessingTracker
    ↓
ConcurrentDictionary
    ↓
frequent independent state updates
```

EXAMPLE / FLOW

```
Gift completed
    ↓
ReceiptQueue
    ↓
bounded Channel<T>
    ↓
BackgroundService
    ↓
Send receipt asynchronously
```

EXAMPLE / FLOW

```
Fund rules loaded
    ↓
ImmutableDictionary snapshot
    ↓
Atomic publication
    ↓
Many request readers
```

Collection Decision Flow

EXAMPLE / FLOW

```
Need a collection?
    ↓
Will it be shared by concurrent writers?
```

No:

EXAMPLE / FLOW

```
Use ordinary generic collection
unless immutability has other value
```

Yes:

EXAMPLE / FLOW

```
What semantics?
```

Key/value:

EXAMPLE / FLOW

```
ConcurrentDictionary
```

FIFO:

EXAMPLE / FLOW

```
ConcurrentQueue
or Channel for async pipeline
```

LIFO:

EXAMPLE / FLOW

```
ConcurrentStack
```

Unordered arbitrary item:

EXAMPLE / FLOW

```
ConcurrentBag
```

Immutable Decision Flow

EXAMPLE / FLOW

```
Does state change rarely
but get read frequently?
    ↓
Yes
```

Ask:

EXAMPLE / FLOW

```
Would readers benefit from
stable versions/snapshots?
```

Yes:

EXAMPLE / FLOW

```
ImmutableArray

ImmutableDictionary

ImmutableHashSet

etc.
```

Then:

EXAMPLE / FLOW

```
One writer?
    →
simple replacement may suffice
```

EXAMPLE / FLOW

```
Many concurrent writers?
    →
ImmutableInterlocked
or explicit synchronization
```

Producer/Consumer Decision Flow

EXAMPLE / FLOW

```
Need producers and consumers?
    ↓
Synchronous/blocking design?
```

Yes:

EXAMPLE / FLOW

```
BlockingCollection<T>
```

Async design:

EXAMPLE / FLOW

```
Channel<T>
```

Need capacity limit:

EXAMPLE / FLOW

```
Use bounded collection/channel
```

This provides backpressure instead of letting producers grow memory usage without limit.

Final End-to-End Mental Model

EXAMPLE / FLOW

```
Normal collection
    ↓
Best when one owner controls mutation
```

EXAMPLE / FLOW

```
Concurrent collection
    ↓
One shared collection
+
multiple concurrent operations
```

EXAMPLE / FLOW

```
Immutable collection
    ↓
No existing instance is mutated
+
readers hold stable versions
```

But always remember:

EXAMPLE / FLOW

```
Thread-safe collection
    ≠
thread-safe values
```

EXAMPLE / FLOW

```
Thread-safe operation
    ≠
atomic business workflow
```

EXAMPLE / FLOW

```
Read-only
    ≠
immutable
```

EXAMPLE / FLOW

```
In-process concurrent collection
    ≠
distributed consistency
```

EXAMPLE / FLOW

```
GetOrAdd factory called
    ≠
factory result definitely inserted
```

EXAMPLE / FLOW

```
Immutable instance safe to share
    ≠
concurrent variable replacement
automatically safe
```

The core principle is:

> **Choose the concurrency model before choosing the collection. Use concurrent collections when multiple threads genuinely need to mutate shared collection state. Use immutable collections when you can avoid mutation and share stable snapshots instead. Prefer atomic collection APIs over check-then-act sequences, keep mutable values and multi-step business invariants in mind, and use async producer/consumer primitives such as `Channel<T>` when the workflow itself is asynchronous.**

For the Fundraising Platform, a practical default is:

EXAMPLE / FLOW

```
Request-local temporary collection
    →
List / Dictionary
```

EXAMPLE / FLOW

```
Shared concurrent key/value cache
    →
ConcurrentDictionary
```

EXAMPLE / FLOW

```
Async background work queue
    →
Channel<T>
```

EXAMPLE / FLOW

```
Read-heavy configuration/reference snapshot
    →
ImmutableArray / ImmutableDictionary
```

CONCURRENT / IMMUTABLE

EXAMPLE / FLOW

```
Cross-server consistency
    →
Database/distributed infrastructure,
not an in-memory collection
```

Next roadmap topic: Topic 30 — CLR, Garbage Collection, and Memory Management.